

PLUGFORGE

Building Developer-First Platforms with API-as-Contract Discipline

Background

Every successful SaaS company eventually faces the same hinge moment: stop being an app, start being a platform. Stripe, Slack, GitHub, Shopify, Linear, Notion — each became dramatically more valuable the day a third-party developer could build on top of them. The disciplines that move a system across that line are not optional: a versioned public API (Stripe's `/v1/`, GitHub's REST and GraphQL surfaces), OAuth 2.0 with PKCE and granular scopes (the entire IETF RFC 6749 + 7636 + 8628 family), signed webhooks with retry and replay semantics (Stripe's `t=/v1=` signature scheme is the de-facto standard), and a typed SDK that makes integration pleasant rather than painful (Stripe's `stripe-node`, GitHub's `Octokit`). The platform companies that get this right become infrastructure. The ones that don't become Zapier sources.

You will take a working open-source collaborative document platform — "Ship" — and add the surfaces a third-party developer would need to build on it. That means a versioned public REST API at `/api/v1/` with consistent error shape and cursor pagination; OAuth 2.0 with Authorization Code + PKCE for web apps and Device Authorization Grant for the CLI; HMAC-signed webhooks with exponential backoff, dead-letter queues, and replay; a typed TypeScript SDK published as `@ship/sdk`; a minimal developer portal in the existing Ship UI; and at least one working reference integration (a CLI tool is must-ship). The architectural payoff lands in the final epic: the Part 2 agent is rewired to authenticate as a first-party OAuth app and consume the public API through the SDK — same scopes, same rate limits, same audit trail. The agent stops being a privileged insider and becomes a platform citizen. The grade is not in the number of endpoints. The grade is in whether a developer can go from `npm install @ship/sdk` to a verified signed webhook in their terminal in under 30 minutes, on a clean machine, following only the published docs.

Project Overview

One-week sprint with four deadlines:

Checkpoint	Deadline
Architectural Defense	Monday 1:00 PM CT
MVP	Tuesday 11:59 PM CT
Early Submission	Friday 11:59 PM CT
Final Submission	Sunday 11:59 AM CT

MVP Requirements

Hard gate. All items required to pass:

- ☐ OAuth app registration endpoint working: admin can create an app, receive a `client_id`, and a `client_secret` hashed in the database (raw secret shown exactly once on creation).
- ☐ Authorization Code + PKCE flow completes end-to-end via a Playwright test: `/oauth/authorize` → consent → `/oauth/token` → usable access token.
- ☐ Bearer token middleware validates tokens on every `/api/v1/*` route; invalid tokens return 401, missing tokens return 401, expired tokens return 401 with a distinct error code.
- ☐ At least one resource (documents) implements GET list, GET by id, and POST. Each route declares its required scope via a `require(scope)` middleware factory.
- ☐ Consistent `ApiError` shape (`{code, message, details?, request_id}`) returned on every public failure, asserted by a fitness test over all `/api/v1` routes.
- ☐ `ScopeRegistry` has `scopes-as-data`; insufficient scope returns 403 with the missing scope named explicitly in the error body (no opaque "forbidden").
- ☐ OpenAPI 3.1 spec served at `/api/v1/openapi.json`, generated from route metadata (never hand-written), validating against the OpenAPI schema in a unit test.
- ☐ SDK skeleton exists in a pnpm workspace package; ``new ShipClient({ token }).me()`` against a running server returns the typed authenticated user.
- ☐ Existing Playwright regression suite passes on main; P95 latency, bundle size, and per-route query counts within +10% of the Part 1 baseline.
- ☐ Deployed and publicly accessible: deployed Ship + published OpenAPI spec URL + at least one OAuth app pre-registered with read-only scopes for graders.

A small public API that matches its spec beats a sprawling public API that contradicts it.

Core Technical Requirements

OAuth + Public API Contract Layer

Feature	Requirements
OAuth App Model	<code>oauth_apps</code> table with <code>id</code> , <code>client_id</code> , hashed <code>client_secret</code> , <code>redirect_uris</code> , <code>owner</code> , <code>requested_scopes</code> . Raw secret shown once on creation and rotation; never recoverable thereafter.
Authorization Code + PKCE	<code>code_challenge</code> and <code>code_challenge_method</code> recorded at <code>/oauth/authorize</code> ; <code>code_verifier</code> required at <code>/oauth/token</code> . Mismatched verifier returns 400 with <code>invalid_grant</code> .
Device Authorization Grant	<code>/oauth/device/code</code> issues a <code>user_code</code> and <code>device_code</code> ; <code>/oauth/device/verify</code> accepts the <code>user_code</code> ; the client polls <code>/oauth/token</code> until authorized. Slow-down responses honored.
Scope Registry	Scopes-as-data: <code>documents:read</code> , <code>documents:write</code> , <code>issues:read</code> , <code>issues:write</code> , <code>sprints:read</code> , <code>sprints:write</code> , <code>webhooks:manage</code> . New scopes register at module load, never edit middleware.

Feature	Requirements
Token Middleware	Bearer validation; populates request with app, user, granted scopes. Invalid token: 401. Insufficient scope: 403 with missing scope named.
Refresh Tokens	One-time-use refresh tokens with rotation. Stolen-refresh-token detection: reuse invalidates the family.
Public API Boundary	Public routes live only at /api/v1/*. Internal endpoints stay at /api/. Lint rule fails the build if a public route imports from internal handler files.
Consistent Error Shape	ApiError { code, message, details?, request_id }. Error middleware ensures every public failure ships this shape. Fitness test verifies.
Cursor Pagination	Opaque base64 cursors over { id, timestamp }. List responses always return { data, next_cursor }. Cursors are stable across reordering operations.
OpenAPI 3.1 Spec	Generated from route metadata in-process. Served at /api/v1/openapi.json. Validates against the OpenAPI schema in a unit test. Spec parity asserted by fitness test.

Webhooks: Signing, Retries, Replay

Feature	Requirements
Event Registry	Event types as data: document.created, document.updated, document.deleted, issue.created, issue.assigned, issue.status_changed, sprint.started, sprint.completed. Each with a Zod schema.
Event Bus	IEventBus interface. Domain layer publishes on writes — never the route layer. In-process implementation must-ship; queue-backed implementation is a Liskov-substitutable drop-in.
Webhook Subscriptions	Per-app per-event-type subscriptions. Target URL, hashed signing secret, active flag. Manageable via /api/v1/webhooks (gated by webhooks:manage scope).
HMAC-SHA256 Signing	Stripe-style header: Ship-Signature: t=<unix-seconds>,v1=<hex-hmac>. Timestamp prevents replay; SDK rejects any signature older than 5 minutes by default.
Retry Schedule	Exponential backoff with jitter: 1s, 4s, 16s, 1m, 5m, 30m. Subscribers returning 5xx or timing out are retried; 4xx responses are treated as permanent failures and dead-lettered.
Dead-Letter Queue	After 6 failed attempts, deliveries land in a DLQ visible in the developer portal. Operators can replay manually; replays carry the original idempotency key.

Feature	Requirements
Delivery Log	webhook_deliveries table records every attempt with subscription_id, event_id, attempt_number, response_status, response_excerpt, latency_ms. Queryable per app.
Replay	/api/v1/webhooks/deliveries/:id/replay re-emits a logged event. Idempotency-Key header passed through so subscribers can dedupe.

SDK, Rate Limiting, Developer Portal

Feature	Requirements
Typed SDK Surface	@ship/sdk exposes resource clients: client.documents, client.issues, client.sprints, client.webhooks. Method signatures match OpenAPI spec; drift fails CI via a fitness test.
OAuth Helpers	ShipClient.authorizationCodeFlow() and ShipClient.deviceLogin() handle their flows end-to-end. Pluggable ITokenStore (in-memory, file, browser localStorage).
Async-Iterator Pagination	for await (const doc of client.documents.iterate()) walks pages transparently. Cursors handled internally; consumer code never sees them.
Webhook Verifier	verifyWebhook(headers, rawBody, secret) returns true/false in one call. Tampered bodies fail; expired timestamps fail; missing v1 header fails.
Typed Error Union	SDK errors are a discriminated union: { kind: 'auth' 'rate_limit' 'not_found' 'validation' 'server', ... }. Consumers can switch on kind exhaustively.
Rate Limit Enforcement	Per-app and per-token token-bucket limits. Public responses carry X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset; 429 responses carry Retry-After.
Public Audit Trail	Every public API call recorded with timestamp, app client_id, user_id, route, scope used, status, latency. Queryable in the developer portal.
Developer Portal	In-app UI for: listing apps, registering apps, viewing/rotating client_secret (shown once), managing subscriptions, browsing the delivery log, replaying failed deliveries.

Testing Scenarios

We will test:

1. Complete the Authorization Code + PKCE flow in a Playwright test from a registered web app. Confirm that a wrong code_verifier on the token exchange returns invalid_grant (negative case is mandatory, not optional).

2. Run the Device Authorization Grant flow from a test CLI: poll /oauth/token until authorized, verify slow-down responses are honored, confirm the resulting token works against /api/v1/me.
3. Enumerate every /api/v1/* route in a fitness test and assert each one (a) has an OpenAPI entry, (b) declares a scope, (c) returns the ApiError shape on failure paths, and (d) supports cursor pagination if it's a list endpoint.
4. Validate the generated /api/v1/openapi.json against the OpenAPI 3.1 JSON schema. Then walk every spec method and assert the SDK exposes a typed call for it.
5. Create a webhook subscription via the SDK; create a document; verify a signed POST arrives at the target URL within 2s; verify the signature with the SDK helper; tamper with the body and verify the helper rejects it.
6. Make a subscriber return 500 on the first three attempts and 200 on the fourth. Verify the retry schedule (1s, 4s, 16s ≥ wait times before each attempt) and that the fourth attempt records success in the delivery log.
7. Force 6 consecutive failures. Verify the delivery lands in the dead-letter queue and is visible in the developer portal. Click "Replay" against a now-healthy subscriber and verify the replay succeeds with the original idempotency key intact.
8. Run the Time-to-First-Event drill end-to-end (see Signature Challenge): from a clean container, pnpm install @ship/sdk → ship login → create document → receive verified webhook in under 30 minutes elapsed (in practice, seconds).

Performance Targets

Metric	Target
Time-to-First-Event (clean machine, docs only)	≤ 30 min real elapsed; CI typically < 60 s
OAuth Auth Code + PKCE round-trip (P95)	< 3 s
OpenAPI spec parity (fitness test)	100% (spec ↔ routes, no drift)
Webhook delivery latency (P95, first attempt)	< 2 s
Webhook retry success rate after transient 5xx	100% within configured schedule
Public API responses with rate-limit headers	100%
Telemetry / regression vs Part 1 baseline	≤ +10% on P95, bundle size, query counts

Signature Challenge: The Time-to-First-Event Drill

This is the project's signature technical challenge and the moment the three-part arc clicks shut. A platform is judged not by the size of its surface but by how quickly a stranger can compose a useful loop on top of it. The Time-to-First-Event (TTFE) drill measures exactly that: on a clean container, with only the published docs and the SDK, how long does it take a developer to go from nothing to a verified signed webhook in their terminal? Anything over 30 minutes means the platform is a curl tutorial. Under 60 seconds in CI means the contract holds end-to-end.

The Five-Line Developer Story

The headline narrative for the demo. The CI drill is a scripted version of this loop:

```
$ pnpm install @ship/sdk
$ ship login                # Device flow
$ ship docs create --title "hello"  # Uses the SDK under the hood
$ ship webhooks tail         # Streams signed deliveries to stdout
→ document.created event arrives, signature verified ✓
```

Required Capabilities

- CLI drill harness: pnpm drill ttfw runs the full loop end-to-end against a containerized Ship instance from a clean working directory.
- Timing instrumentation: each stage of the drill (install, login, register subscription, create document, receive webhook, verify signature) records elapsed milliseconds.
- Signature verification by the SDK in one line — `verifyWebhook(headers, rawBody, secret)`. Tampered or expired payloads must fail; valid payloads must pass.
- Drill runs in CI on every PR. Any regression past the configured threshold fails the build.

Interface Definitions

```
// Consistent error shape across all /api/v1 responses
interface ApiError {
  code: "unauthorized" | "forbidden" | "not_found"
    | "validation_failed" | "rate_limited" | "server_error";
  message: string;
  details?: Record<string, unknown>;
  request_id: string;
}

// SDK client surface – resource-segregated (ISP)
class ShipClient {
  readonly documents: DocumentsClient;
  readonly issues: IssuesClient;
  readonly sprints: SprintsClient;
  readonly webhooks: WebhooksClient;

  static async deviceLogin(opts: {
    onUserCode: (code: string, verifyUrl: string) => void;
    tokenStore?: ITokenStore;
  }): Promise<ShipClient>;
}

// Webhook signature: Stripe-style timestamp + HMAC
// Header: Ship-Signature: t=1715985600,v1=<hex-hmac-sha256>
function verifyWebhook(
  headers: Record<string, string>,
  rawBody: string,
  secret: string,
  toleranceSec?: number, // default 300
): boolean;
```

Example Drill Loop

```
// integrations/cli/tests/ttfe.drill.ts
test("time to first event", async () => {
  const t0 = performance.now();

  // 1. Fresh client, device login
  const client = await ShipClient.deviceLogin({
    onUserCode: (code) => process.env.SHIP_DEVICE_CODE = code,
  });

  // 2. Subscribe to document.created
  const sub = await client.webhooks.create({
    event: "document.created",
    target_url: testListener.url,
  });

  // 3. Trigger the event
  const doc = await client.documents.create({ title: "hello" });

  // 4. Wait for signed delivery
  const delivery = await testListener.waitFor(
    (h, body) => verifyWebhook(h, body, sub.signing_secret),
    { timeoutMs: 5000 },
  );

  expect(delivery.event.type).toBe("document.created");
  expect(performance.now() - t0).toBeLessThan(60_000);
});
```

Evaluation Criteria — Drill Stages to Expected Outcomes

Stage	Expected Outcome
Install (pnpm install @ship/sdk)	Workspace package resolves; types load in editor; no peer-dependency errors
Auth (ship login via device flow)	User code displayed; polling succeeds within 60s in tests; token persists in configured store
Subscribe (client.webhooks.create)	Subscription persisted; signing secret returned once; subscription appears in dev portal
Trigger (client.documents.create)	Document created; document.created event published on the bus; subscribers receive POST
Verify (verifyWebhook helper)	Valid signature passes; tampered body fails; timestamp older than 5 min fails
Total elapsed	< 60 s in CI; ≤ 30 min on a clean machine following only the published docs

Implement at Least 5 of the Following Integrations / Flows

- ☐ CLI tool with device flow — ship login, ship docs ls/get/create, ship webhooks tail (must-ship).

- ☐ Slack integration — receives signed webhooks, posts document.created and issue.assigned to channels via Slack OAuth (should-ship).
- ☐ Browser SDK demo — Authorization Code + PKCE in a single-page app that lists the user's documents.
- ☐ GitHub integration — links Ship issues to GitHub PRs via webhook + GitHub App.
- ☐ Refresh-token rotation drill — proves a stolen refresh token, when reused, invalidates the entire family.
- ☐ Idempotency-Key end-to-end — replay drill that confirms subscribers correctly dedupe on replayed deliveries.
- ☐ In-process plugin runtime (stretch) — isolated-vm with one hook (document.beforeCreate) and a hard CPU/memory cap; explicitly experimental.

Performance Targets — Signature Challenge

Metric	Target
TTFE drill runtime in CI (P95)	< 60 s
TTFE on a clean machine (docs only)	≤ 30 min real elapsed
Webhook signature verification (SDK helper)	< 1 ms per call
Drill flake rate over 20 consecutive CI runs	0% (any flake = bug in the drill or the platform)
SDK install size (production deps only)	< 250 KB minified + gzipped

AI Cost Analysis

The headline cost discipline: the platform itself does zero AI work. The LLM is invoked only on user-initiated agent turns — exactly as in Part 2. The architectural payoff of Epic 7 is that the agent runs through the public API like any other client, so its cost shape is unchanged but its access shape is now the same as an external developer's. Cost scales with agent activity, not platform traffic.

Development & Testing Costs to Track

- LLM API spend during the agent rewire (Epic 7) — track per-day spend while migrating direct service calls to SDK calls; confirm the rewire does not change token volume.
- CI minutes for the TTFE drill — every PR runs the full end-to-end loop. Time it on Day 1 and budget the weekly CI bill explicitly.
- OAuth flow testing — Playwright browser launches for the auth-code flow consume CI compute. Count them.
- OpenAPI spec generation and validation overhead in CI — small, but worth a number rather than a hand-wave.
- Storage and egress for the dev portal demo — webhook delivery logs grow with every drill run; size them at the expected demo volume.

Production Cost Projections

Platform-layer cost scales with API traffic and webhook delivery, not with LLM calls. Numbers below assume the agent app is one of N installed apps at each tier; LLM cost is attributable to the agent app's user-driven sessions, not the platform itself.

Tier	API calls/day	Webhook deliveries/day	Agent LLM calls/day	Est. cost/month
100 users	~20,000	~5,000	~50	\$2–8
1,000 users	~200,000	~50,000	~500	\$15–50
10,000 users	~2,000,000	~500,000	~5,000	\$80–250
100,000 users	~20,000,000	~5,000,000	~50,000	\$500–1,500

Include Assumptions

- Webhook fanout ratio — number of webhook deliveries triggered per write operation, given the average number of subscriptions per event type at each tier. State this explicitly.
- Agent active rate — fraction of users who actually use agent features on a given day, and average agent turns per active user. Cost projection bends on this assumption, not on platform traffic.
- Storage retention — delivery log rows × retention days × bytes per row, plus audit log rows. State both retention windows and explain why each is set there.

Technical Stack

Layer	Technology
Backend	Node.js + Express (existing Ship stack); TypeScript strict mode required; Zod for request/response schemas and OpenAPI generation.
Frontend (Dev Portal)	React (existing Ship UI); the portal reuses the public API like any other client (eat the dog food).
AI / LLM (agent only)	Claude API (Sonnet 4 recommended), OpenAI GPT-4 class, or local Llama via Ollama. The platform itself is LLM-free.
OAuth Implementation	Hand-rolled minimal IETF-correct flows (RFC 6749 + 7636 PKCE + 8628 Device Grant) for learning; alternatives include node-oauth2-server, Ory Hydra, or Auth0 fronting Ship.
Webhook Queue	In-memory must-ship; alternatives include BullMQ + Redis, Inngest, or AWS SQS for production; signed payloads compatible with all of them.
Rate Limiting	Token-bucket in-memory must-ship; alternatives include @upstash/ratelimit, Redis-backed token bucket, or Cloudflare rate-limit rules at the edge.
OpenAPI / SDK	Zod schemas → OpenAPI 3.1 via zod-to-openapi or @asteasolutions/zod-to-openapi; SDK hand-written in TypeScript for quality, fitness-tested against the spec for parity.

Layer	Technology
Reference Integrations	CLI in Node + commander or oclif; Slack integration in Express + @slack/bolt; GitHub integration via @octokit/auth-app (stretch).
Patterns	SOLID via TS interfaces and a single composition root; public/internal API boundary enforced by workspace lint rules; in-memory test doubles for every interface.
Deployment	Fly.io, Railway, Render, or AWS for Ship; @ship/sdk published as a workspace package (npm-publish documented but not required for the week).

Use whatever stack helps you ship. Complete the Pre-Search process to make informed decisions.

Build Strategy

Priority Order

1. OAuth foundation FIRST. Without working tokens and scope checks, nothing else has a contract. Get Authorization Code + PKCE end-to-end against a Playwright-driven browser on Day 1 — negative tests (wrong verifier rejected) included. Device Authorization Grant follows the same day.
2. Public/internal API boundary on Day 1. Create /api/v1/ as a fresh router that does NOT share middleware with the internal API. Add the lint rule that fails the build on cross-imports before you have any cross-imports to lint. This decision is far cheaper to enforce than to retrofit.
3. Error shape and ApiError class before any resource endpoint. Every /api/v1 failure must ship the same shape. Build the fitness test that enumerates routes and asserts the shape — that's your TODO list for E2.
4. OpenAPI generated from route metadata, never hand-written. Get the generator working end-to-end with one resource (documents) before adding issues, sprints, and me. The fitness test that asserts spec ↔ route parity is the single best defense against drift.
5. Webhooks end-to-end on Day 4: event registry → event bus → subscriptions → signer → queue deliverer → delivery log → replay. All seven slices in one day is aggressive but tractable because each one is small. The signer (HMAC-SHA256 with Stripe-style timestamp) has its own unit test suite — positive, negative, replay, tamper.
6. SDK skeleton + one resource client + auth helpers next. Iterate by having the CLI (E6) consume the SDK as you build it. The SDK's worst bugs always surface when an actual consumer compiles against it.
7. CLI reference integration (must-ship). The CLI is the proof the platform works. ship login (device flow), ship docs create (write through SDK + public API), ship webhooks tail (the demo moment).
8. Developer portal + agent rewire (Epic 7). Portal is should-ship and short — it consumes the public API like any other client. The agent rewire is the architectural payoff: replace direct service calls with SDK calls, behind a feature flag so Part 2's tests pass with the flag on or off.

Critical Guidance

- Public/internal split is a one-way door. If you let routes from /api/ leak into /api/v1/ "just this once," you have permanently damaged the contract. The lint rule is not optional.
- Generate the OpenAPI spec; do not write it. Hand-written specs lie within a week. Every public route's request/response schema lives in Zod adjacent to the handler; the generator walks them.

- Webhook in-memory deliverer for unit tests resolves synchronously. The real queue-backed deliverer is tested with deterministic clock injection — never with `setTimeout` waits in tests. Timing-based webhook tests are flaky tests.
- One LLM call per agent turn, period. The platform never invokes the LLM. If you find yourself wanting platform-layer AI features ("smart suggestions for OAuth scopes"), you're scope-creeping.
- External integrations live in integrations/ and import only @ship/sdk — never api/src/. Enforced by a workspace dependency rule. This is what makes "the agent is a platform citizen" true rather than aspirational.
- Time-to-first-event drill in CI from Day 5 onward. Once the SDK and one resource exist, the drill exists. It will catch contract regressions faster than any unit test.

Required Documentation

Architecture Document (1–2 pages) committed at docs/architecture.md.

Section	Content
Module Layout	Tree of api/src/platform/ and sdk/ with one sentence per module (apps, oauth, scopes, ratelimit, webhooks, api/v1, openapi, audit).
SOLID Rationale	One paragraph per principle showing exactly where it appears in your code, with a file path reference. ScopeRegistry as OCP, IEventBus as DIP, resource-segregated SDK clients as ISP are strong candidates.
Composition Root	Annotated pseudo-code of api/src/app.ts wiring concrete OAuth, rate-limiter, event-bus, and webhook-deliverer implementations. Include the in-memory test wiring as a sibling diagram.
Public/Internal Boundary	Sequence diagram showing how /api/v1/ routes call the same domain services as internal routes, with auth/scope/audit/webhook attaching only at the public layer.
OAuth Flows	Sequence diagrams for Authorization Code + PKCE and Device Authorization Grant. Mark where PKCE verifier is validated and where refresh-token rotation happens.
Webhook Pipeline	Event source → IEventBus → subscription matcher → signer → IWebhookDeliverer → retry scheduler → delivery log. Mark where the signature is computed and where Idempotency-Key originates.
SDK Surface	Public surface of @ship/sdk: resource clients, auth helpers, async iterators, error union, webhook verifier. Mark which surfaces are stable and which are pre-1.0.
Agent-as-Citizen	Before/after diagram of the agent's call path. Before: direct domain calls. After: OAuth app → SDK → public API → same domain services. Mark the audit-log payoff.

Section	Content
Failure Modes	What happens when: the token store is corrupted, a subscriber's signing secret is rotated mid-flight, the queue deliverer crashes, the OpenAPI generator throws at boot. One paragraph each.

Submission Requirements

Deadline: Sunday 10:59 PM CT

Deliverable	Requirements
GitHub Repository	Public; per-slice branches preserved; each PR description lists which acceptance criterion that slice advances and confirms the fitness test passed.
Demo Video (3–5 min)	The five-line story is the demo: open a fresh terminal → pnpm install @ship/sdk → ship login → ship docs create → ship webhooks tail produces a verified signed delivery. Then switch to the dev portal and replay one delivery.
Pre-Search Document	All three phases completed with written answers; saved AI conversation attached as a reference artifact.
Architecture Document	1–2 pages following the Section/Content table above. Committed at docs/architecture.md.
OpenAPI Spec	Live at /api/v1/openapi.json on the deployed instance, plus a static copy at docs/openapi.json in the repo. Validate against the OpenAPI schema.
AI Cost Analysis	Tracked dev spend, production projections table, explicit assumptions for webhook fanout, agent active rate, and storage retention.
Per-Epic Write-up	Before → fix → after → proof. For Epic 6, proof is the TTFE drill passing in CI. For Epic 7, proof is the agent's audit-log rows showing OAuth app authentication.
Three Discoveries	Strong candidates: OAuth Device Authorization Grant in TypeScript, Zod-driven OpenAPI generation with fitness-test parity, Stripe-style HMAC + timestamp anti-replay, async-iterator pagination as a developer-experience pattern.
Deployed Application	Public URL with a pre-registered OAuth app (read-only scopes) for graders, plus credentials in the README. Dev portal reachable; OpenAPI spec resolvable.
Social Post	Tag @GauntletAI. The screenshot is the ship webhooks tail terminal showing a verified signed event arriving in real time.

Interview Preparation

Technical Topics

- Walk me through how a request authenticated by your platform reaches a domain service. Where exactly do AuthN, AuthZ, rate-limit, audit, and webhook publication attach? Why is each a separate middleware?
- Why Authorization Code + PKCE for web apps and Device Authorization Grant for the CLI, instead of the same flow for both? What does PKCE buy you that client_secret doesn't?
- Show me your webhook signature scheme. Why a timestamp in the header — what attack does it prevent, and what is your tolerance window? What happens if your server's clock drifts?
- How do you stop the OpenAPI spec from drifting from the actual server behavior? Walk me through the fitness test and what it would fail on.
- The agent is now a platform citizen — it goes through the SDK and OAuth like any external app. What did this cost you in code, and what did it buy you architecturally?
- Sketch the path from the user creating a document in the UI to a Slack message appearing in a channel. Name every interface boundary the event crosses.

Mindset & Growth

- Which slice taught you the most about API design? What did you ship that you'd shape differently if you started over?
- Where did you cut scope this week, and how did you decide? What was the smallest version you would still call "a platform"?
- What contract decision (error shape, pagination, signing scheme) do you regret most? Why does it matter past Week 6?
- Walk through a bug the TTFE drill caught that your unit tests missed. What changed in your test discipline afterward?

Final Note

A small public API that matches its spec beats a sprawling public API that contradicts it. One excellent reference integration beats three half-finished ones. An agent that goes through the front door beats an agent with a privileged shortcut.

Depth over breadth. Proof over promises. The TTFE drill is the rubric.

Gate: Project completion + interviews required for Austin admission. The interview is where you defend your public/internal boundary, your OAuth flow choices, and the moment the agent becomes a citizen of the platform it lives in.

Appendix: Pre-Search Checklist

Complete this before writing code. Save your AI conversation as a reference document and attach it to your final submission.

Phase 1: Define Your Constraints

1.1 — Scale & Load Expectations

- What is the realistic API request rate against your deployed instance during the demo window, and how does that map to webhook fanout (one document.created can produce N deliveries given N matching subscriptions)?
- How many OAuth apps and subscriptions will you seed for the grader? At what fanout does your in-memory deliverer start dropping below the < 2 s P95 target?
- How many concurrent CLI sessions will run device flow during a demo, and does your polling-rate response (slow_down semantics) handle them correctly?
- What is your delivery-log row growth rate at the demo's expected event rate, and how long is the log retained?

1.2 — Budget & Cost Ceilings

- What is your weekly LLM budget for the Epic 7 agent rewire? The rewire shouldn't change token volume — how do you verify that with a before/after measurement?
- What is your daily ceiling on CI minutes given that every PR runs the TTFE drill plus the OAuth Playwright flow plus the full regression suite?
- What is the SDK install footprint budget you're committing to — production deps only, gzipped — and how will you enforce it (bundle analyzer, CI size check)?
- If your webhook deliverer's queue runs away (a subscriber that 5xx's forever multiplied by every event), what is your runaway-cost ceiling and what mechanism enforces it?

1.3 — Timeline & Scope Reality

- Which of E1–E7 are must-ship for you given your OAuth experience? Which reference integration is your must-ship — CLI (recommended), Slack (more visual), or something else?
- How many hours per day will you actually spend on this — be honest. What does your day-by-day plan look like against that number?
- What is your kill criterion for the developer portal? If E5 is taking too long, is read-only delivery-log-viewer the minimum viable portal?

1.4 — Security & Data Sensitivity

- Where do client_secret values live at rest — hashed with what algorithm, salted how, recoverable via what process if a user loses theirs?
- How long are access tokens valid, and what is your refresh-token rotation policy? Will you implement stolen-refresh-token detection (reuse invalidates the family)?
- What goes in webhook payloads vs. what gets fetched on demand — do you ship document content in document.created, or just the ID? Defend the tradeoff between subscriber convenience and exposure surface.
- How do you protect the developer portal's secret display (shown-once UX) from accidental leakage via screenshot, log line, or browser back-button?

1.5 — Team Skill Inventory

- Have you implemented OAuth 2.0 end-to-end before, or only consumed it? If only consumed, which morning do you spend on RFC 6749 + 7636 + 8628 before starting E1?
- How comfortable are you with Zod and zod-to-openapi (or equivalent)? Where does your fallback live if generation breaks late in the week?
- Have you designed an SDK before? Have you been on the consuming side of a bad one? Which of those experiences guides your API choices more this week?

Phase 2: Architecture Discovery

2.1 — OAuth Flow Choices

- Will you support refresh tokens from day one, or start with long-lived access tokens and add refresh later? What is the migration cost if you wait?
- How will you handle scope upgrades — does a user who originally granted documents:read need to re-consent to grant documents:write, or do you support incremental consent?
- Where does the consent screen live — a route inside Ship's UI, a dedicated endpoint with its own minimal layout, or something else? What protects it from clickjacking?
- For the Device Authorization Grant: what is your verification URL UX — do users paste a code into a form, or do you embed the code in a URL they click? RFC 8628 allows both.

2.2 — Public API Shape

- Will your error shape match exactly across all routes (one fitness test asserts it), or will some routes carry richer details? If both, where is the line and is it documented?
- How will you handle field-level filtering or sparse fieldsets — query parameters (?fields=...), header (Prefer:), or skip it for the week? Defend the call.
- What is your versioning policy past /api/v1/ — additive only, breaking changes via /v2/, or deprecation headers with sunset dates? Which is in the docs by Sunday?
- Will every list endpoint return cursor pagination, or will small static lists (like /api/v1/scopes) skip it? Where do you draw the line and how does the fitness test know?

2.3 — Webhook Reliability

- What exactly is signed — the raw request body, the body plus the timestamp, the body plus a versioned scheme tag? Why?
- What is your retry schedule (the brief suggests 1s, 4s, 16s, 1m, 5m, 30m) and how is it tested without sleeping in test code? Deterministic clock injection — where does it live?
- How does your deliverer know a subscriber is permanently broken vs transiently? Is 4xx always permanent, 5xx always transient, or is the answer more nuanced (e.g., 410 Gone permanent, 429 transient)?
- How does Idempotency-Key flow from your replay endpoint through to subscribers, and what is the contract you document for subscriber dedupe?

2.4 — SDK Design

- Will your SDK methods be generated from the OpenAPI spec or hand-written and parity-tested against it? Defend the tradeoff between type quality and drift risk.
- What is your error model in the SDK — typed discriminated union (recommended), throw-and-catch with structured errors, or Result-style return? Which feels most TypeScript-native today?

- How does the SDK handle pagination — return raw cursors and let consumers loop, return async iterators only, or both? Async-iterators-only is cleanest; both is more flexible.
- Where does ITokenStore's contract live — does it persist refresh tokens too, or only access tokens? What is the threading model for refresh under concurrent calls?

2.5 — Developer Portal & Self-Service

- Will the portal reuse the public API like any other client, or will it have a privileged internal endpoint for admin operations? Eating the dog food is more rigorous; an internal escape hatch is more pragmatic.
- How is client_secret rotation modeled — is the old secret immediately invalidated, or does it work alongside the new one for a grace period? What does Stripe do, and why?
- How will the delivery-log view scale visually when an app has thousands of deliveries — server-side pagination, virtualized list, time-bucket filters? Which is build-cheap and which is rebuild-cheap later?
- Will the portal show webhook payloads in full, redacted, or behind a click-to-reveal? Defend the choice against the leakage concerns from 1.4.

2.6 — Agent-as-Citizen Rewire

- Which OAuth flow does the agent use — Authorization Code, Device Grant, or Client Credentials (RFC 6749 §4.4) for first-party machine-to-machine? Defend the choice.
- How is the agent's app seeded — at boot, via a migration, manually in dev? What guarantees it exists in deployed environments?
- Which scopes does the agent request, and what is your defense for each? Does the agent need write scopes, or can it stay read-only behind a recommendation pattern?
- Behind a feature flag, both old (direct service calls) and new (SDK calls) paths exist. How does CI prove Part 2's tests pass with the flag both on and off?

Phase 3: Post-Stack Refinement

3.1 — Security & Failure Modes

- What happens when an OAuth app's owner is deleted — apps deactivated, transferred to admin, or orphaned with a soft-flag? Each is a different recovery story.
- What is the failure mode when the webhook deliverer crashes mid-batch — at-least-once delivery (subscribers must dedupe), at-most-once (some lost), or exactly-once aspiration with idempotency keys?
- How do you detect and respond to a leaked client_secret — automatic rotation, manual rotation by the owner, or admin-driven force-rotate? What's the audit signal you'd alert on?
- What is your CSRF protection on the developer portal's app-form and rotate-secret endpoints, given they sit alongside the OAuth consent screen?

3.2 — Testing Strategy

- How is the TTFE drill written — full pnpm install in a fresh container, or workspace symlink with the install step mocked? Which proves more, and which is fast enough for CI?
- How will OAuth Playwright tests stay stable — do you stub Keycloak/external IdPs, or run a containerized auth server? What does the trade cost in CI minutes?
- What is your strategy for testing the webhook deliverer's retry schedule without sleeping in tests? Deterministic clocks, virtual timers, or fast-forward control?

3.3 — Tooling & CI

- Which lint rules catch the public/internal boundary violations early — no imports from `api/src/` in `api/src/platform/api/v1/`, no imports from `api/src/` in `integrations/`, both?
- How will the OpenAPI fitness test be wired into CI — fail the build on drift, or warn and post a diff comment? What about additive changes?
- How will the +10% performance regression budget be enforced — manual benchmark, automated baseline comparison, perf job that fails the PR?

3.4 — Deployment & Hosting

- Where does the deployed Ship instance live, and how do you give graders a pre-registered OAuth app without exposing your tenant's data?
- Will the OpenAPI spec be served from the live instance only, or also published as a static doc (Stoplight, Redoc, Swagger UI) at a stable URL?
- If a grader wants to install the CLI from your repo and run it against your deployed instance, what is the one-command setup, and where does it live in the README?

3.5 — Observability of API Usage

- What metrics do you record per public API call (route, status, latency, scope used, app, user, request_id), and where do they show up (logs, /metrics, dev portal)?
- How will you tell, post-demo, that the agent actually went through the public API for every action — a grep of the audit log, a dashboard panel, or a fitness test that runs the agent and inspects the trail?
- How does Idempotency-Key reuse vs. fresh keys show up in your delivery log? Could you tell whether a subscriber's dedupe is working from your portal alone?